PROJET 4 DATA ANALYST

Réalisez une étude de santé publique avec R ou Python

OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner codes, textes et graphique.

Cet outil est formidable pour plusieurs raisons:

- il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'un instruction, de la corriger si nécessaire.
- De rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- d'agrémenter de graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandée.

Et garder à l'esprit, qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Note jeremy Est ce qu'il faut faire le calcul de la sous nutrition sur les pays qu'on a ? Est ce qu'il faut faire des graphiques ? Rajouter le soja La liste des céréales est difficile a trouver ...

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, ca
```

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

```
#Importation de la librairie Pandas
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

1.2 - Chargement des fichiers Excel

```
#Importation du fichier population.csv
population = pd.read_csv('population.csv')

#Importation du fichier dispo_alimentaire.csv
dispo_alimentaire = pd.read_csv('dispo_alimentaire.csv')

#Importation du fichier aide_alimentaire.csv
aide_alimentaire = pd.read_csv('aide_alimentaire.csv')

#Importation du fichier sous_nutrition.csv
sous_nutrition = pd.read_csv('sous_nutrition.csv')
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier population

```
#Afficher les dimensions du dataset

#population
print("Le tableau population comporte {} observation(s) ou article(s)".format(p
print("Le tableau population comporte {} colonne(s)".format(population.shape[1]
```

```
Le tableau population comporte 1416 observation(s) ou article(s)
Le tableau population comporte 3 colonne(s)
```

```
#Consulter le nombre de colonnes
print("Le tableau population comporte {} colonne(s)".format(population.shape[1]
#La nature des données dans chacune des colonnes
population.info()
#Le nombre de valeurs présentes dans chacune des colonnes
print("Le tableau population comporte {} ligne(s)".format(population.shape[0]))
```

```
Le tableau population comporte 3 colonne(s)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1416 entries, 0 to 1415
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Zone    1416 non-null   object
 1   Année   1416 non-null   int64
 2   Valeur  1416 non-null   float64
dtypes: float64(1), int64(1), object(1)
memory usage: 33.3+ KB
Le tableau population comporte 1416 ligne(s)
```

```
#Affichage les 5 premières lignes de la table
population.head(5)
```

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

Étapes suivantes : Générer du code population | Afficher les graphiques recommandés | Nouv. interactive

```
#Nous allons harmoniser les unités. Pour cela, nous avons décidé de multiplier
#Multiplication de la colonne valeur par 1000
population['Valeur'] = population['Valeur']*1000
```

```
#changement du nom de la colonne Valeur par Population
population = population.rename(columns={'Valeur':'Population'})
```

```
#Affichage les 5 premières lignes de la table pour voir les modifications
population.head(5)
```

	Zone	Année	Population
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

Étapes suivantes : Générer du code population | Afficher les graphiques recommandés | Nouv. interactive

2.2 - Analyse exploratoire du fichier disponibilité alimentaire

```
#Afficher les dimensions du dataset
#dispo_alimentaire
print("Le tableau dispo_alimentaire comporte {} observation(s) ou article(s)".f
print("Le tableau dispo_alimentaire comporte {} colonne(s)".format(dispo_alimer
```

```
Le tableau dispo_alimentaire comporte 15605 observation(s) ou article(s)
Le tableau dispo_alimentaire comporte 18 colonne(s)
```

```
#Consulter le nombre de colonnes
print("Le tableau dispo_alimentaire comporte {} colonne(s)".format(dispo_alimer
```

```
Le tableau dispo_alimentaire comporte 18 colonne(s)
```

```
#Affichage les 5 premières lignes de la table
dispo_alimentaire.head(5)
```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilit alimentair (Kcal/personne/jour
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.

Étapes suivantes : du code dispo_alimentaire graphiques interactive

```
#remplacement des NaN dans le dataset par des 0
dispo_alimentaire = dispo_alimentaire.fillna(0)


#multiplication de toutes les lignes contenant des milliers de tonnes en Kg
dispo_alimentaire[['Aliments pour animaux','Autres Utilisations','Disponibilité
```

```
#Affichage les 5 premières lignes de la table
dispo_alimentaire.head(5)
```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilit alimentair (Kcal/personne/jour
0	Afghanistan	Abats Comestible	animale	0.0	0.0	5.
1	Afghanistan	Agrumes, Autres	vegetale	0.0	0.0	1.
2	Afghanistan	Aliments pour enfants	vegetale	0.0	0.0	1.
3	Afghanistan	Ananas	vegetale	0.0	0.0	0.
4	Afghanistan	Bananes	vegetale	0.0	0.0	4.

Étapes suivantes : du code dispo_alimentaire graphiques interactive

2.3 - Analyse exploratoire du fichier aide alimentaire

```
#Afficher les dimensions du dataset
#aide_alimentaire
print("Le tableau aide_alimentaire comporte {} observation(s) ou article(s)".fo
print("Le tableau aide_alimentaire comporte {} colonne(s)".format(aide_alimenta
```

```
Le tableau aide_alimentaire comporte 1475 observation(s) ou article(s)
Le tableau aide_alimentaire comporte 4 colonne(s)
```

```
#Consulter le nombre de colonnes
print("Le tableau aide_alimentaire comporte {} colonne(s)".format(aide_alimenta
```

```
Le tableau aide_alimentaire comporte 4 colonne(s)
```

```
#Affichage les 5 premières lignes de la table
aide_alimentaire.head(5)
```

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504

Étapes suivantes : du code aide_alimentaire graphiques interactive

```
#changement du nom de la colonne Pays bénéficiaire par Zone
aide_alimentaire = aide_alimentaire.rename(columns={'Pays bénéficiaire':'Zone'}
```

```
#Multiplication de la colonne Aide_alimentaire qui contient des tonnes par 1000
aide_alimentaire['Valeur'] = aide_alimentaire['Valeur']*1000
```

```
#Affichage les 5 premières lignes de la table
aide_alimentaire.head(5)
```

	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

Étapes suivantes : du code aide_alimentaire graphiques interactive

2.3 - Analyse exploratoire du fichier sous nutrition

```
#Afficher les dimensions du dataset
#sous_nutrition
print("Le tableau sous_nutrition comporte {} observation(s) ou article(s)".form
print("Le tableau sous_nutrition comporte {} colonne(s)".format(sous_nutrition.
```

```
Le tableau sous_nutrition comporte 1218 observation(s) ou article(s)
Le tableau sous_nutrition comporte 3 colonne(s)
```

```
#Consulter le nombre de colonnes
print("Le tableau sous_nutrition comporte {} colonne(s)".format(sous_nutrition.
```

```
Le tableau sous_nutrition comporte 3 colonne(s)
```

```
#Afficher les 5 premières lignes de la table
sous_nutrition.head(5)
```

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

Étapes suivantes : Générer du code sous_nutrition | Afficher les graphiques recommandés | Nouveau interactive

```
#Conversion de la colonne sous nutrition en numérique
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'], errors='coer
```

```
#Conversion de la colonne (avec l'argument errors=coerce qui permet de converti
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'],errors ='coer
#Puis remplacement des NaN en 0
sous_nutrition = sous_nutrition.fillna(0)
```

```
#changement du nom de la colonne Valeur par sous_nutrition
sous_nutrition = sous_nutrition.rename(columns={'Valeur':'sous_nutrition'})
```

```
#Multiplication de la colonne sous_nutrition par 1000000
sous_nutrition['sous_nutrition'] = sous_nutrition['sous_nutrition']*1000000
```

```
#Afficher les 5 premières lignes de la table
sous_nutrition.head(5)
```

	Zone	Année	sous_nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

Étapes suivantes : Générer du code sous_nutrition | Afficher les graphiques recommandés | Nouveau interactive

3.1 - Proportion de personnes en sous nutrition

```
# Il faut tout d'abord faire une jointure entre la table population et la table

population_2017 = population.loc[population['Année'] == 2017, :]
sous_nutrition_2017 = sous_nutrition.loc[sous_nutrition['Année'] == '2016-2018'

population_join_sous_nutrition_2017 = pd.merge(population_2017, sous_nutrition_
population_join_sous_nutrition_2017 = population_join_sous_nutrition_2017.fillr
```

```
population_join_sous_nutrition_2017 = population_join_sous_nutrition_2017.renam
population_join_sous_nutrition_2017 = population_join_sous_nutrition_2017.renam
population_join_sous_nutrition_2017 = population_join_sous_nutrition_2017.renam
population_join_sous_nutrition_2017 = population_join_sous_nutrition_2017.renam
population_join_sous_nutrition_2017.head(5)
```

	Zone	Année_population	Population	Année_sous_nutrition	sous_nutri
0	Afghanistan	2017	36296113.0	2016-2018	10500(
1	Afrique du Sud	2017	57009756.0	2016-2018	3100(
2	Albanie	2017	2884169.0	2016-2018	100(
3	Algérie	2017	41389189.0	2016-2018	1300(
4	Allemagne	2017	82658409.0	2016-2018	

Étapes suivantes : du code `population_join_sous_nutrition_2017` graphiques inte

```
#Affichage du dataset
population_join_sous_nutrition_2017.head(5)
```

	Zone	Année_population	Population	Année_sous_nutrition	sous_nutri
0	Afghanistan	2017	36296113.0	2016-2018	10500(
1	Afrique du Sud	2017	57009756.0	2016-2018	3100(
2	Albanie	2017	2884169.0	2016-2018	100(
3	Algérie	2017	41389189.0	2016-2018	1300(
4	Allemagne	2017	82658409.0	2016-2018	

Étapes suivantes : du code `population_join_sous_nutrition_2017` graphiques inte

```
#Calcul et affichage du nombre de personnes en état de sous nutrition
sum_population_join_sous_nutrition_2017 = sum(population_join_sous_nutrition_20
print(sum_population_join_sous_nutrition_2017, 'personnes sont en sous nutritic
```

```
535700000.0 personnes sont en sous nutrition
```

```
#Somme population totale (pour voir population totale VS population en sous nut
sum_population = sum(population_join_sous_nutrition_2017['Population'])
print('la population totale est de', sum_population, 'personnes')
```

```
la population totale est de 7543798779.0 personnes
```

```
part_sous_nutrition = sum_population_join_sous_nutrition_2017 / sum_population
print('le pourcentage de personne en etat de sous nutriton est de', round(part_
```

```
le pourcentage de personne en etat de sous nutriton est de 7.1 %
```

3.2 - Nombre théorique de personne qui pourrait être nourries

```
#Combien mange en moyenne un être humain ? Source =>
#1800 a 2700 kcal par jour / selon https://www.fao.org/
moyenne_conso_humain = np.mean([1800, 2700])
print('un humain mange en moyenne', moyenne_conso_humain, 'kcal')
```

```
un humain mange en moyenne 2250.0 kcal
```

```
#On commence par faire une jointure entre le data frame population et Dispo_ali
population_dispo_alimentaire_2017 = pd.merge(population.loc[population['Année']
population_dispo_alimentaire_2017 = population_dispo_alimentaire_2017.rename(cc
population_dispo_alimentaire_2017 = population_dispo_alimentaire_2017.fillna(0)
```

```
#Affichage du nouveau dataframe
population_dispo_alimentaire_2017.head(5)
```

	Zone	Année	Population	Produit	Origine	Aliments pour animaux	Autres Utilisations	(
0	Afghanistan	2017	36296113.0	Abats Comestible	animale	0.0	0.0	
1	Afghanistan	2017	36296113.0	Agrumes, Autres	vegetale	0.0	0.0	
2	Afghanistan	2017	36296113.0	Aliments pour enfants	vegetale	0.0	0.0	
3	Afghanistan	2017	36296113.0	Ananas	vegetale	0.0	0.0	
4	Afghanistan	2017	36296113.0	Bananes	vegetale	0.0	0.0	

Étapes suivantes : du code population_dispo_alimentaire_2017 graphiques intera

```
#Création de la colonne dispo_kcal avec calcul des kcal disponibles mondialemer
population_dispo_alimentaire_2017['dispo_kcal'] = population_dispo_alimentaire_
#somme dispo kcal par jour et par personne
print('il y a', population_dispo_alimentaire_2017['dispo_kcal'].sum(), 'kcal di
```

```
il y a 20918984627331.0 kcal dispo par jour en 2017
```

```
#Affichage du nouveau dataframe
population_dispo_alimentaire_2017.head(5)
```

	Zone	Année	Population	Produit	Origine	Aliments pour animaux	Autres Utilisations	(
0	Afghanistan	2017	36296113.0	Abats Comestible	animale	0.0	0.0	
1	Afghanistan	2017	36296113.0	Agrumes, Autres	vegetale	0.0	0.0	
2	Afghanistan	2017	36296113.0	Aliments pour enfants	vegetale	0.0	0.0	
3	Afghanistan	2017	36296113.0	Ananas	vegetale	0.0	0.0	
4	Afghanistan	2017	36296113.0	Bananes	vegetale	0.0	0.0	

5 rows × 21 columns

```
#Calcul du nombre d'humains pouvant être nourris
#division du nombre de kcal dispo mondialement par le nombre de kcal que consom
humains_nourris = (population_dispo_alimentaire_2017['dispo_kcal'].sum() * 365)
print('on peut nourrir',round(humains_nourris), 'humains en 2017')
```

```
#Pour expliquer le résultat dans la présentation à Marc, calculs supplémentaire
#1.calories par humain

calories_par_humain = calories_par_humain_par_jour = population_dispo_alimentai
print('par jour et par humain =', calories_par_humain, 'kcal')

#2.calcul nombre de kcal en 2017
print('il y a', (population_dispo_alimentaire_2017['dispo_kcal'].sum() * 365),
```

```
on peut nourrir 9297326501 humains en 2017
par jour et par humain = 2773.004058056809 kcal
il y a 7635429388975815.0 kcal disponible pour toute la planete en 2017
```

3.3 - Nombre théorique de personne qui pourrait être nourrie avec les produits végétaux

```
#Transfert des données avec les végétaux dans un nouveau dataframe
vegetaux = population_dispo_alimentaire_2017.loc[population_dispo_alimentaire_2
```

```
#Calcul du nombre de kcal disponible pour les végétaux
sum_vegetaux = vegetaux['dispo_kcal'].sum()
print(sum_vegetaux, 'kcal sont disponibles pour les végétaux')
```

```
17260764211501.0 kcal sont disponibles pour les végétaux
```

```
#Calcul du nombre de kcal végétales dispo par humain et par jour
print('il y a', vegetaux['dispo_kcal'].sum() / sum_population, 'kcal végétales
```

```
il y a 2288.073252901514 kcal végétales disponibles par jour et pour chaque
```

```
#Calcul du nombre d'humains pouvant être nourris avec les végétaux
humain_nourris_vegetaux = (sum_vegetaux * 365) / (moyenne_conso_humain * 365)
print(round(humain_nourris_vegetaux), 'humains pourraient être nourris avec les

#pour présentation à marc calculer le nombre de kcal necessaire pour un humain
print('a l annee un humain doit consommer',(moyenne_conso_humain * 365), 'kcal'
```

```
7671450761 humains pourraient être nourris avec les végétaux
a l annee un humain doit consommer 821250.0 kcal
```

3.4 - Utilisation de la disponibilité intérieure

```
#Calcul de la disponibilité totale
disponibilite_totale = dispo_alimentaire['Disponibilité intérieure'].sum()
print('La disponibilité totale est de', disponibilite_totale, 'milliers de tonr
```

```
La disponibilité totale est de 9848994000.0 milliers de tonnes
```

```
#création d'une boucle for pour afficher les différentes valeurs en fonction de

liste_produit = ['Aliments pour animaux', 'Pertes', 'Nourriture', 'Semences', '
for part in liste_produit:
  print(dispo_alimentaire[part].sum() * 100 / disponibilite_totale, '%')
```

```
13.242418464261426 %
4.606541541196999 %
49.51021393657058 %
1.5705258831511117 %
22.384895350733284 %
8.782856401374596 %
```

3.5 - Utilisation des céréales

```
#Création d'une liste avec toutes les variables
liste_cereales = ['Blé', 'Riz (Eq Blanchi)', 'Orge', 'Maïs', 'Millet', 'Seigle'
```

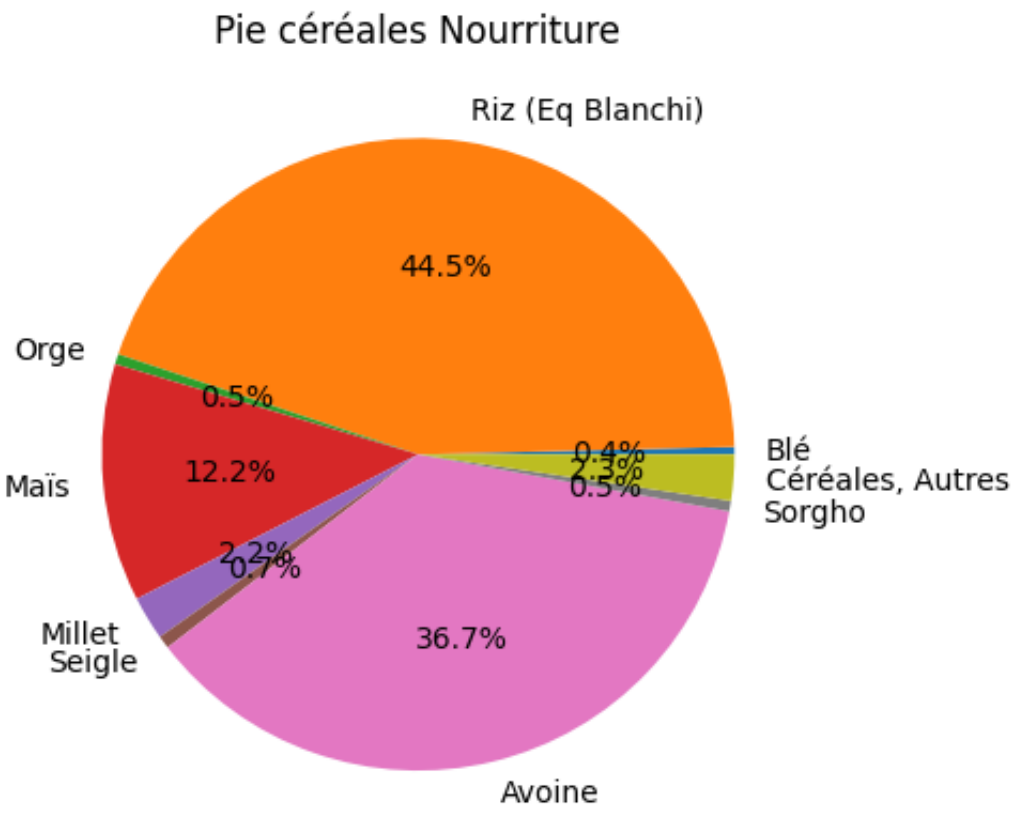
```
#Création d'un dataframe avec les informations uniquement pour ces céréales
cereales = dispo_alimentaire.loc[dispo_alimentaire['Produit'].isin(liste_cereal
cereales.head()

cereales = cereales.groupby(['Produit']).sum()
```

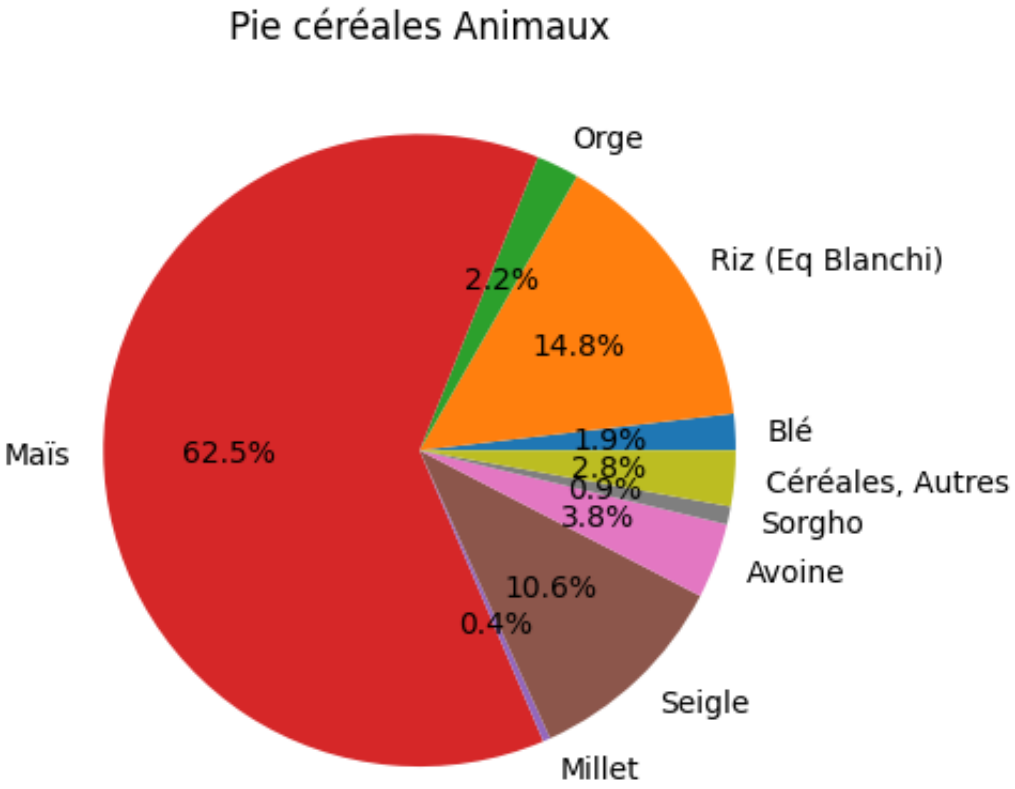
```
#Affichage de la proportion d'alimentation animale
cereales_quantite_aliment_animale = sum(dispo_alimentaire['Aliments pour animau
dispo_interieure = sum(dispo_alimentaire['Disponibilité intérieure'])
print('La proportion de céréales pour l alimentation végétale est de', round(ce
```

```
La proportion de céréales pour l alimentation végétale est de 13 %
```

```
# Pie céréales Nourriture

y = np.array(cereales['Nourriture'])

mylabels = ['Blé', 'Riz (Eq Blanchi)', 'Orge', 'Maïs', 'Millet', 'Seigle', 'Avo

plt.pie(y, labels = mylabels, autopct='%1.1f%%')
plt.title('Pie céréales Nourriture')
plt.show()
```

```
# Pie céréales Animaux


y = np.array(cereales['Aliments pour animaux'])

mylabels = ['Blé', 'Riz (Eq Blanchi)', 'Orge', 'Maïs', 'Millet', 'Seigle', 'Avc

plt.pie(y, labels = mylabels, autopct='%1.1f%%')
plt.title('Pie céréales Animaux')
plt.show()
```


Pie céréales Nourriture
Riz (Eq Blanchi)
44.5%
Orge
0.5%
Maïs
12.2%
0.4%
2.3%
0.5%
Blé
Céréales, Autres
Sorgho
2.2%
0.7%
Millet
Seigle
36.7%
Avoine

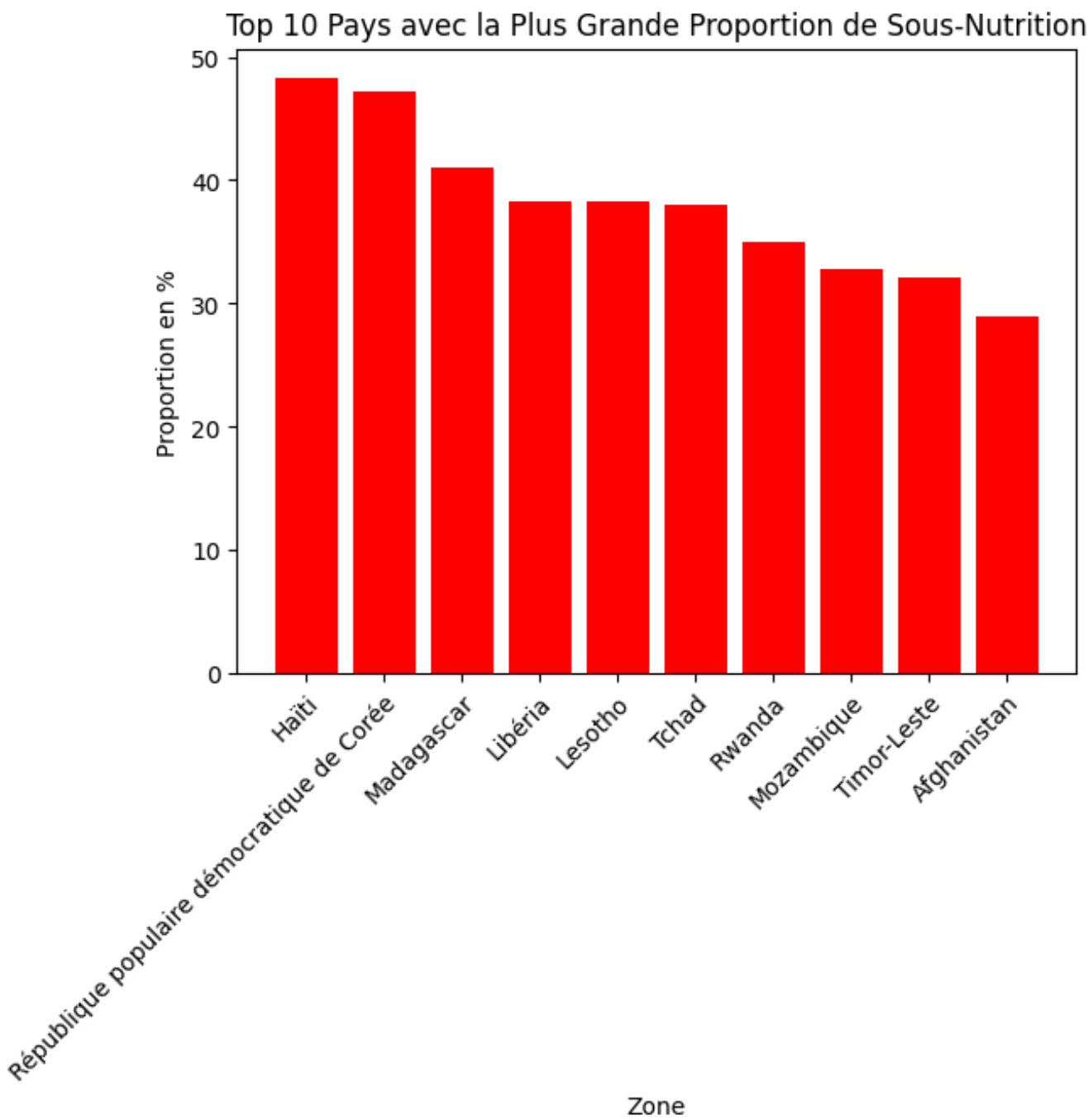

Pie céréales Animaux
Orge
2.2%
Riz (Eq Blanchi)
14.8%
Maïs
62.5%
1.9%
Blé
2.8%
0.9%
Céréales, Autres
Sorgho
3.8%
Avoine
10.6%
0.4%
Seigle
Millet

3.6 - Pays avec la proportion de personnes sous-alimentée la plus forte en 2017

```
#Création de la colonne proportion par pays
population_join_sous_nutrition_2017['proportion en %'] = population_join_sous_r
```

```
#affichage après trie des 10 pires pays
top10_pays_sous_nutrition = population_join_sous_nutrition_2017.sort_values(by=
```

```
x = top10_pays_sous_nutrition['Zone']
y = top10_pays_sous_nutrition['proportion en %']

plt.bar(x, y, color = "red")
plt.xlabel('Zone')
plt.ylabel('Proportion en %')
plt.title('Top 10 Pays avec la Plus Grande Proportion de Sous-Nutrition')
plt.xticks(rotation=45, ha="right", rotation_mode="anchor")

plt.show()
```



3.7 - Pays qui ont le plus bénéficié d'aide alimentaire depuis 2013

```
#calcul du total de l'aide alimentaire par pays
aide_alimentaire_par_pays = aide_alimentaire.groupby(['Zone'], as_index=False)|
aide_alimentaire_par_pays.head(10)
```

	Zone	Valeur
0	Afghanistan	185452000
1	Algérie	81114000
2	Angola	5014000
3	Bangladesh	348188000
4	Bhoutan	2666000
5	Bolivie (État plurinational de)	6000
6	Burkina Faso	64812000
7	Burundi	77318000
8	Bénin	22224000
9	Cambodge	25780000

Étapes suivantes : du code aide_alimentaire_par_pays graphiques interactive

```
#affichage après trie des 10 pays qui ont bénéficié le plus de l'aide alimentai
top10_pays_aide_alimentaire = aide_alimentaire_par_pays.sort_values(by='Valeur'
top10_pays_aide_alimentaire.head()
```

	Zone	Valeur
50	République arabe syrienne	1858943000
75	Éthiopie	1381294000
70	Yémen	1206484000
61	Soudan du Sud	695248000
60	Soudan	669784000

Étapes suivantes : du code top10_pays_aide_alimentaire graphiques interactive

3.8 - Evolution des 5 pays qui ont le plus bénéficiés de l'aide alimentaire entre 2013 et 2016

```
#Création d'un dataframe avec la zone, l'année et l'aide alimentaire puis group
zone_annee_aide_alimentaire_2013_2016 = aide_alimentaire.loc[(aide_alimentaire[
zone_annee_aide_alimentaire_2013_2016 = aide_alimentaire.groupby(['Zone','Année
```

```
#Création d'une liste contenant les 5 pays qui ont le plus bénéficiées de l'aic
top5_aide_alimentaire_2013_2016 = zone_annee_aide_alimentaire_2013_2016.groupby
top5_aide_alimentaire_2013_2016 = top5_aide_alimentaire_2013_2016.sort_values(b
top5_aide_alimentaire_2013_2016.head(5)
top5_aide_alimentaire_2013_2016 = top5_aide_alimentaire_2013_2016['Zone'].tolis
print(top5_aide_alimentaire_2013_2016)
```

```
['République arabe syrienne', 'Éthiopie', 'Yémen', 'Soudan du Sud', 'Soudan
```

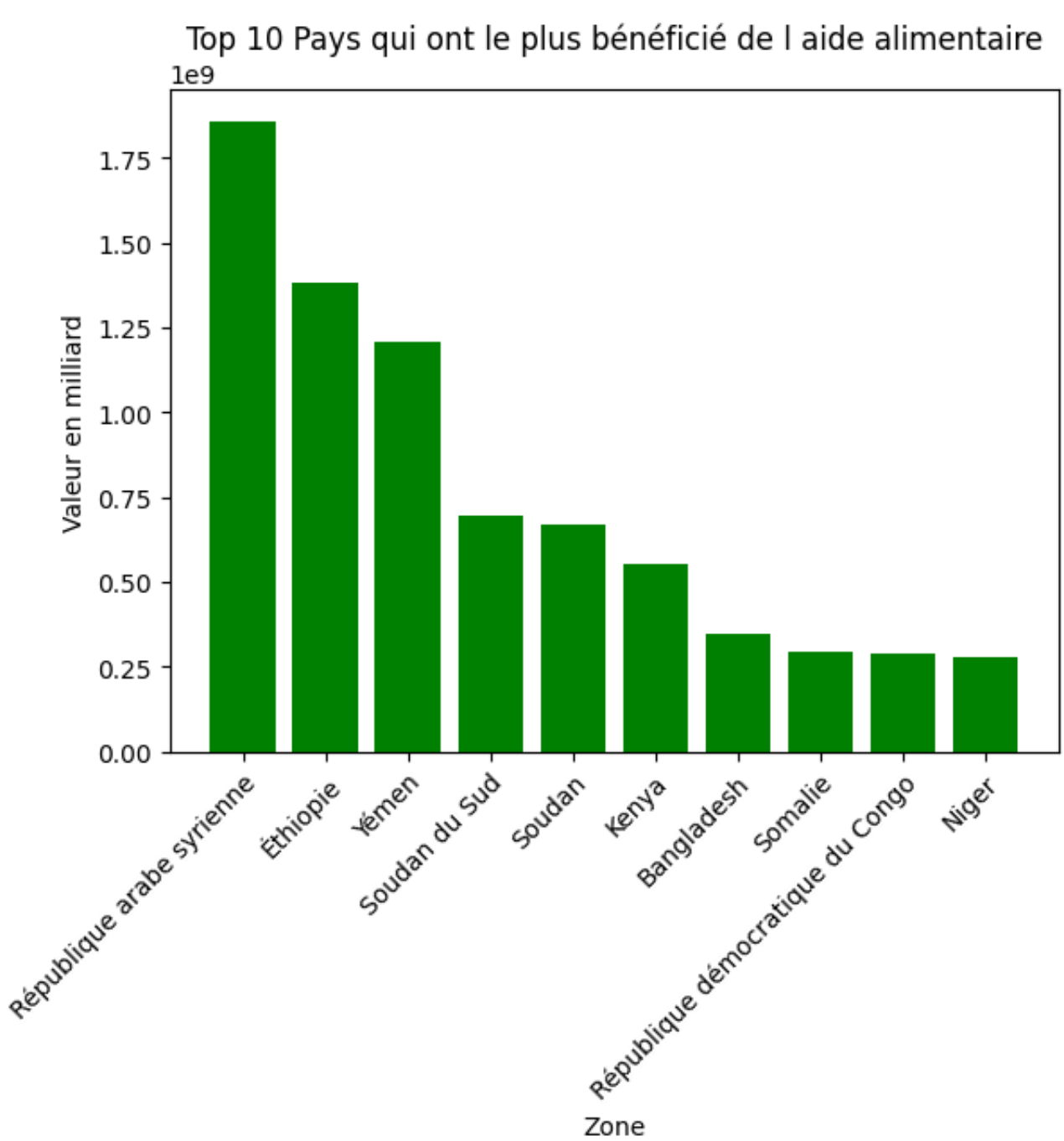
Générer randomly select 5 items from a list Fermer

```
#Création d'une liste contenant les 10 pays qui ont le plus bénéficiées de l'ai
top10_aide_alimentaire_2013_2016 = zone_annee_aide_alimentaire_2013_2016.groupb
top10_aide_alimentaire_2013_2016 = top10_aide_alimentaire_2013_2016.sort_values
top10_aide_alimentaire_2013_2016.head(10)
```

	Zone	Valeur
50	République arabe syrienne	1858943000
75	Éthiopie	1381294000
70	Yémen	1206484000
61	Soudan du Sud	695248000
60	Soudan	669784000
30	Kenya	552836000
3	Bangladesh	348188000
59	Somalie	292678000
53	République démocratique du Congo	288502000
43	Niger	276344000

Étapes suivantes : du code `top10_aide_alimentaire_2013_2016` graphiques interac

```
x = top10_aide_alimentaire_2013_2016['Zone']
y = top10_aide_alimentaire_2013_2016['Valeur']

plt.bar(x, y, color = "green")
plt.xlabel('Zone')
plt.ylabel('Valeur en milliard')
plt.title('Top 10 Pays qui ont le plus bénéficié de l aide alimentaire')
plt.xticks(rotation=45, ha="right", rotation_mode="anchor")

plt.show()
```



```
#On filtre sur le dataframe avec notre liste
zone_annee_aide_alimentaire_2013_2016 = zone_annee_aide_alimentaire_2013_2016.l
```

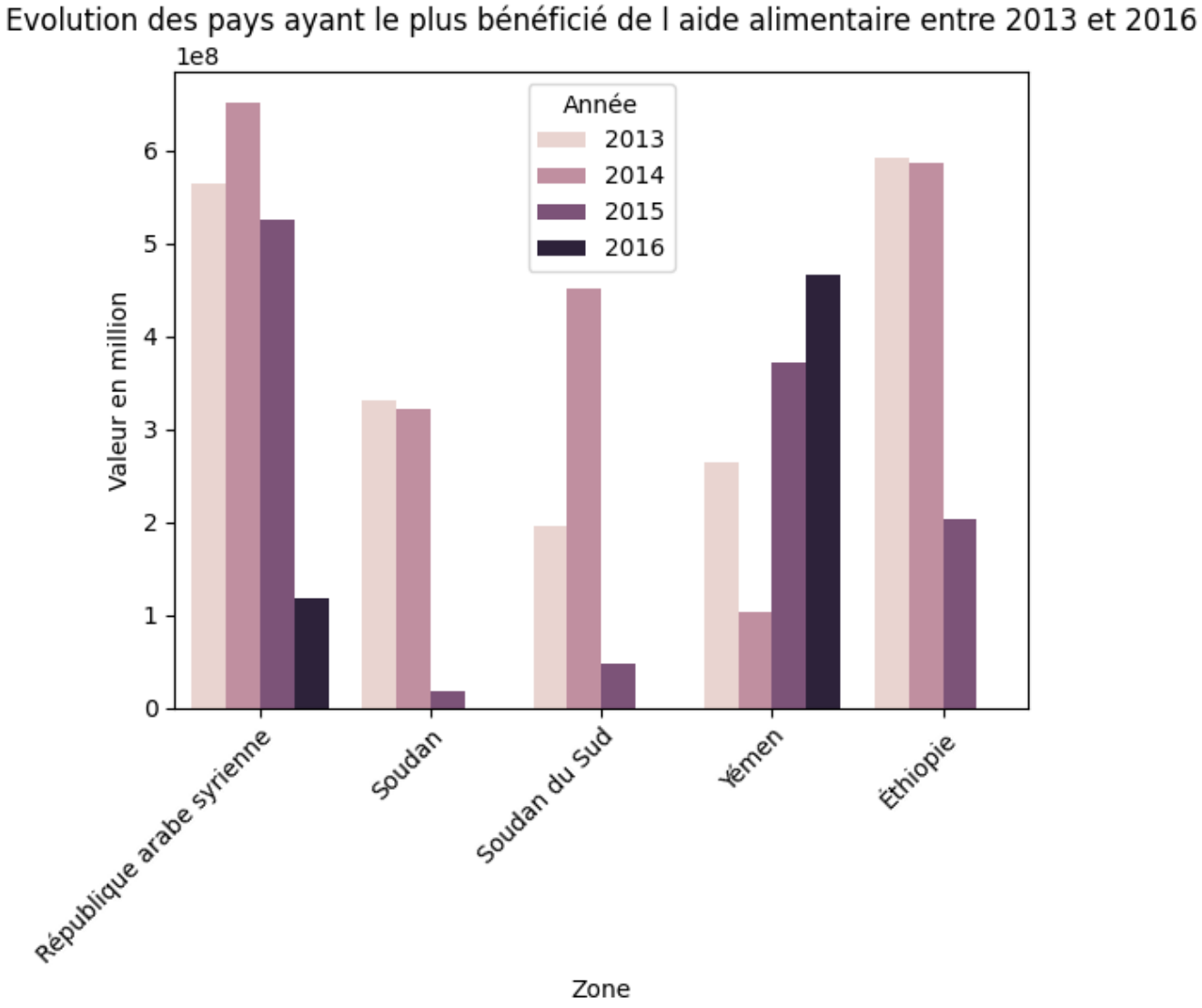
```
# Affichage des pays avec l'aide alimentaire par année
zone_annee_aide_alimentaire_2013_2016.head(30)
```

	Zone	Année	Valeur
157	République arabe syrienne	2013	563566000
158	République arabe syrienne	2014	651870000
159	République arabe syrienne	2015	524949000
160	République arabe syrienne	2016	118558000
189	Soudan	2013	330230000
190	Soudan	2014	321904000
191	Soudan	2015	17650000
192	Soudan du Sud	2013	196330000
193	Soudan du Sud	2014	450610000
194	Soudan du Sud	2015	48308000
214	Yémen	2013	264764000
215	Yémen	2014	103840000
216	Yémen	2015	372306000
217	Yémen	2016	465574000
225	Éthiopie	2013	591404000
226	Éthiopie	2014	586624000
227	Éthiopie	2015	203266000

Étapes suivantes : du code zone_annee_aide_alimentaire_2013_2016 graphiques

```
sns.barplot(data=zone_annee_aide_alimentaire_2013_2016, x='Zone', y='Valeur', c

plt.ylabel('Valeur en million', fontsize=10)
plt.title('Evolution des pays ayant le plus bénéficié de l aide alimentaire ent
plt.xticks(rotation=45, ha="right", rotation_mode="anchor")
```

```
<ipython-input-64-387d6bc35b2d>:1: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

  sns.barplot(data=zone_annee_aide_alimentaire_2013_2016, x='Zone', y='Vale
([0, 1, 2, 3, 4],
 [Text(0, 0, 'République arabe syrienne'),
  Text(1, 0, 'Soudan'),
  Text(2, 0, 'Soudan du Sud'),
  Text(3, 0, 'Yémen'),
  Text(4, 0, 'Éthiopie')])
```



3.9 - Pays avec le moins de disponibilité par habitant

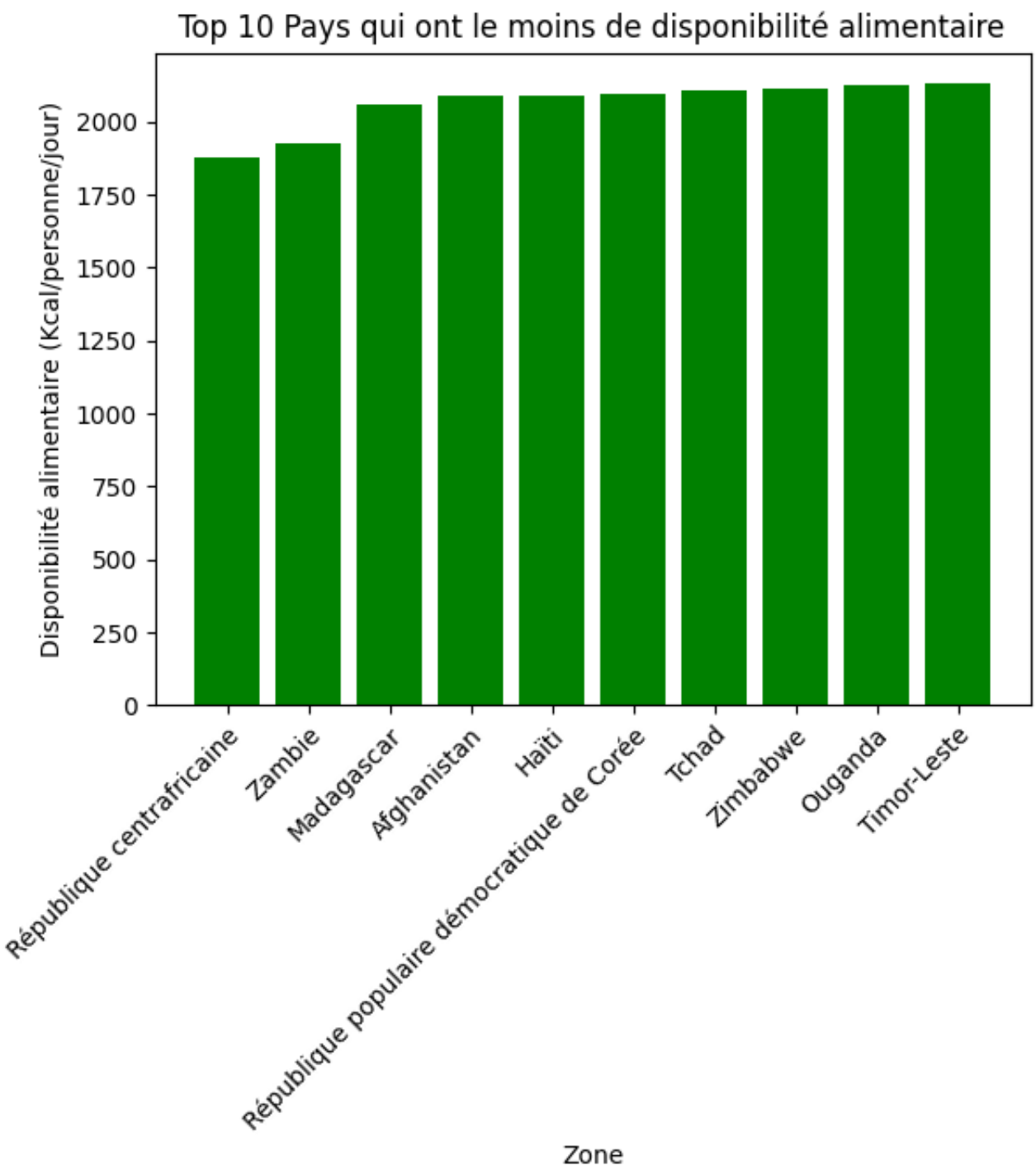
```
#Calcul de la disponibilité en kcal par personne par jour par pays
dispo_kcal_personne_jour = dispo_alimentaire.groupby(['Zone'], as_index=False)|
dispo_kcal_personne_jour.head(10)
```

	Zone	Disponibilité alimentaire (Kcal/personne/jour)
0	Afghanistan	2087.0
1	Afrique du Sud	3020.0
2	Albanie	3188.0
3	Algérie	3293.0
4	Allemagne	3503.0
5	Angola	2474.0
6	Antigua-et-Barbuda	2416.0
7	Arabie saoudite	3255.0
8	Argentine	3226.0
9	Arménie	2924.0

Étapes suivantes : du code dispo_kcal_personne_jour graphiques interactive

```
#Affichage des 10 pays qui ont le moins de dispo alimentaire par personne
top10_moins_de_dispo_alimentaire_2013_2016 = dispo_kcal_personne_jour.sort_valu
top10_moins_de_dispo_alimentaire_2013_2016 = top10_moins_de_dispo_alimentaire_2
```

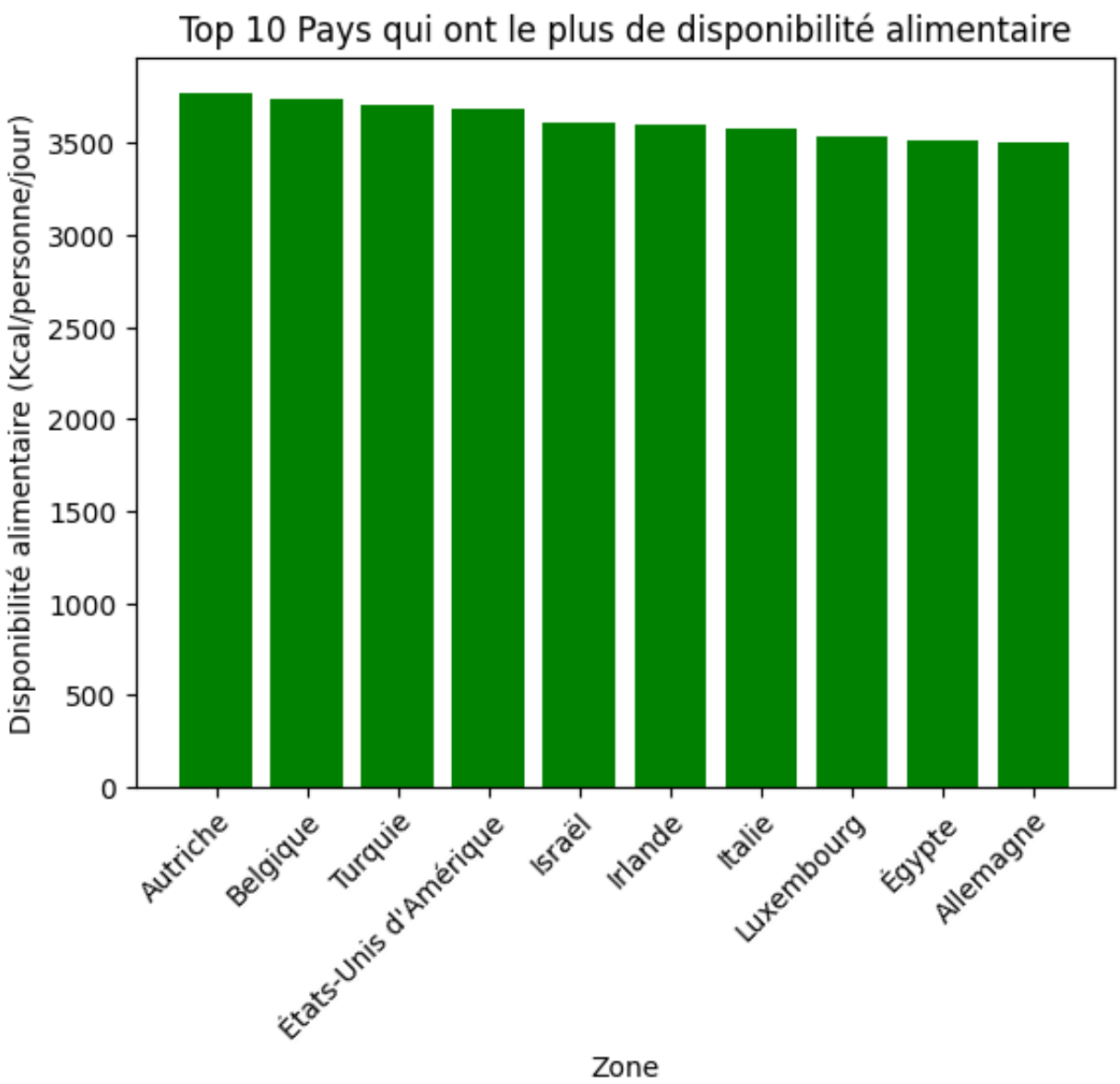
```
x = top10_moins_de_dispo_alimentaire_2013_2016['Zone']
y = top10_moins_de_dispo_alimentaire_2013_2016['Disponibilité alimentaire (Kcal

plt.bar(x, y, color = "green")
plt.xlabel('Zone')
plt.ylabel('Disponibilité alimentaire (Kcal/personne/jour)')
plt.title('Top 10 Pays qui ont le moins de disponibilité alimentaire')
plt.xticks(rotation=45, ha="right", rotation_mode="anchor")

plt.show()
```



3.10 - Pays avec le plus de disponibilité par habitant

```
#Affichage des 10 pays qui ont le plus de dispo alimentaire par personne
top10_dispo_alimentaire_2013_2016 = dispo_kcal_personne_jour.sort_values(by='Di
```

```
x = top10_dispo_alimentaire_2013_2016['Zone']
y = top10_dispo_alimentaire_2013_2016['Disponibilité alimentaire (Kcal/personne

plt.bar(x, y, color = "green")
plt.xlabel('Zone')
plt.ylabel('Disponibilité alimentaire (Kcal/personne/jour)')
plt.title('Top 10 Pays qui ont le plus de disponibilité alimentaire')
plt.xticks(rotation=45, ha="right", rotation_mode="anchor")

plt.show()
```



3.11 - Exemple de la Thaïlande pour le Manioc

```
#création d'un dataframe avec uniquement la Thaïlande
data_thailande = dispo_alimentaire.loc[dispo_alimentaire['Zone'] == 'Thaïlande'
```

```
#Calcul de la sous nutrition en Thaïlande
data_thailande_sous_nutrition = sous_nutrition.loc[(sous_nutrition['Zone'] == '
population_thailande = population.loc[(population['Zone'] == 'Thaïlande') & (pc

proportion_sous_nutrition_thailande = data_thailande_sous_nutrition['sous_nutri
print('La proportion de personne en sous nutrition en thailande est de', round(
```

```
La proportion de personne en sous nutrition en thailande est de 8.96 %
```

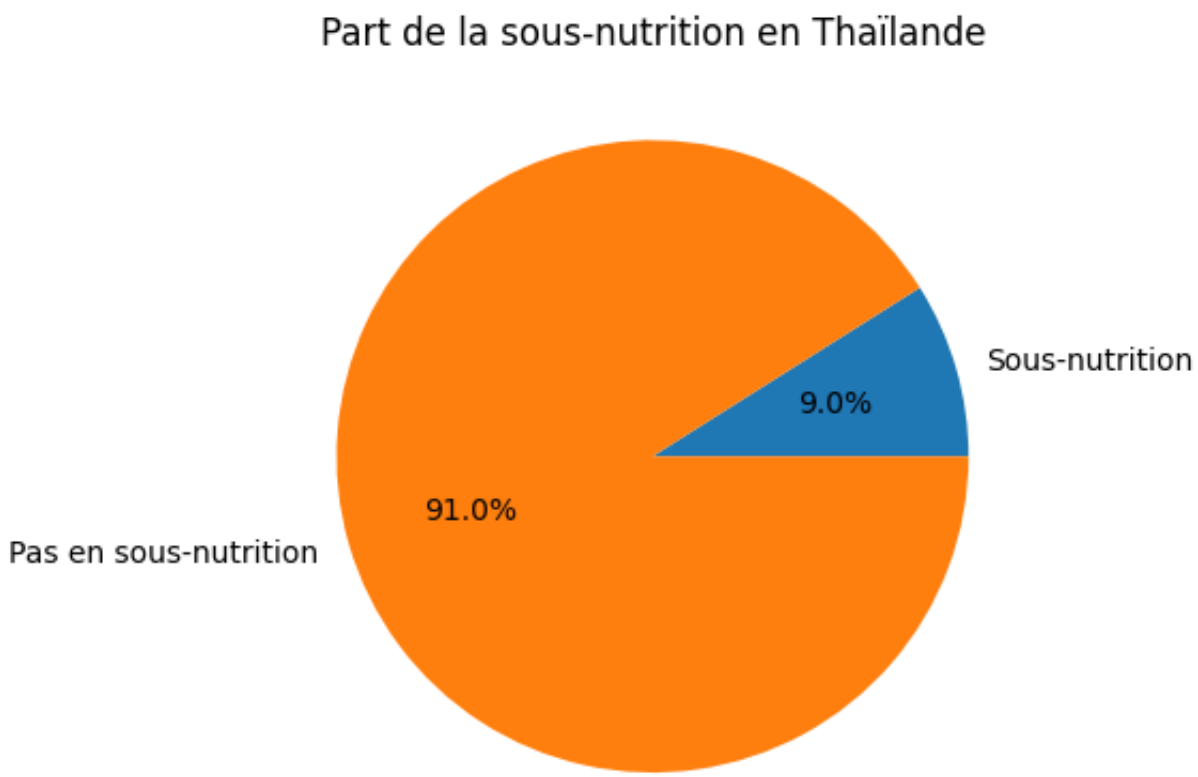
```
# On calcule la proportion exportée en fonction de la proportion
proportion_export = data_thailande['Exportations - Quantité'].sum() / data_thai
print('la proportion de produits exportés sur la disponibilité totale de produi

# On calcule la proportion exportée de manioc en fonction de la proportion de c
proportion_export_manioc = data_thailande.loc[data_thailande['Produit'] == 'Mar
print('la proportion de manioc exportée est de',round(proportion_export_manioc,

#Pas logique => plus d'exportation que de disponibilité
```

```
la proportion de produits exportés sur la disponibilité totale de produits
la proportion de manioc exportée est de 402.52 % ce qui n est pas logique,
```

```
#Calcul du nombre de kcal de manioc disponibles par jour et par personne en tha
disponibilité_manioc_par_personne = data_thailande.loc[data_thailande['Produit'
print('la disponibilité alimentaire en kcal de manioc par personne est de', dis

#Calcul du nombre de kcal total disponibles par jour et par personne en thailar
disponibilité_kcal_thailande_par_personne = data_thailande['Disponibilité alime
print('la disponibilité de kcal totale par personne en thailande est de', dispo
```

```
la disponibilité alimentaire en kcal de manioc par personne est de 40.0 kca
la disponibilité de kcal totale par personne en thailande est de 2785.0 kca
```

```
#Réalisation d'un graphique pour la planche de présentation à Marc
# Calcul de la part "pas en sous nutrition en thailande pour réaliser la pie"
proportion_thailande_pas_sous_nutrition = 100 - proportion_sous_nutrition_thaila

labels = ['Sous-nutrition', 'Pas en sous-nutrition']
sizes = [proportion_sous_nutrition_thailande, proportion_thailande_pas_sous_nutr

plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.title("Part de la sous-nutrition en Thaïlande")
plt.show()
```



Etape 6 - Analyse complémentaires

```
#Quelle est la part d'exportation / importations de produits en haiti qui pourr

dispo_alimentaire_Haiti = dispo_alimentaire.loc[dispo_alimentaire['Zone'] == 'H
dispo_intérieure_Haiti = dispo_alimentaire_Haitin['Disponibilité intérieure'].s

liste_utilisation = ['Disponibilité intérieure', 'Exportations - Quantité', 'Im
for part in liste_utilisation:
  print(dispo_alimentaire_Haiti[part].sum() * 100 / dispo_intérieure_Haiti, '%'

#=> Repartition qui a l'air en adequation avec leur situation
```

```
100.0 %
0.242914979757085 %
19.497975708502025 %
66.41295546558705 %
10.995951417004049 %
82.13765182186235 %
0.582995951417004 %
12.34008097165992 %
```

```
#Combien de personnes pourrait-on nourrir en Haiti en 2017 avec la quantité de


#Calcul kcal dispo en viandes pour tous les types de viandes par jour et par pe
disponibilité_viande_monde_2017 = dispo_alimentaire.loc[(dispo_alimentaire['Pro


#population haiti en 2017
population_haiti_2017 = population.loc[(population['Zone'] == 'Haïti') & (popul
print(population_haiti_2017, 'habitants en Haiti en 2017')

#Calories necessaires à nourrir Haiti en 2017
Calories_necessaires_pour_haiti_2017 = population_haiti_2017 * moyenne_conso_hu
print(Calories_necessaires_pour_haiti_2017, 'kcal sont nécessaires pour nourrir
```

```
10982366.000000002 habitants en Haiti en 2017
9019268077500.002 kcal sont nécessaires pour nourrir Haiti en 2017
```

```
#Calcul personnes pouvant être nourries avec la viande en Haiti en 2017
personnes_nourries_viande_2017 = (disponibilité_viande_monde_2017 * 365) / (moy

print(round(personnes_nourries_viande_2017,2), 'personnes pourraient être nourr
```

```
18.38 personnes pourraient être nourries avec la viande mondiale en Haiti e
```